

Lab 2: Multi-Page Product SPA

Props, Emits, Vue Router & Lifecycle Hooks

Objective

Evolve your Lab 1 product page into a real multi-page SPA. You will apply proper component structure, routing, props and emits communication, and lifecycle hooks throughout the entire app.

Rules Before You Start

Views vs Components — Non Negotiable

Every page that responds to a route goes in /views. Every reusable UI piece goes in /components. If it has a route, it is a View. If it does not, it is a Component. No exceptions.

Data Lives in App.vue Only

All product data is defined in App.vue and passed down via props. No data defined inside any view or component. Props down. Emits up. Always.

Navigation — No <a> Tags

Never use <a> tags for internal navigation. Use RouterLink in templates. Any method that navigates without a full page reload is acceptable in script.

Lifecycle Hooks — Every Single File

Every view and every component must have both onMounted and onUnmounted with a console.log. Open your browser console while navigating — you should see mounts and unmounts firing. If you do not see them, something is wrong.

File Structure

```
src/  
├─ views/
```

```
├── HomeView.vue
├── ProductView.vue
├── AboutView.vue
├── components/
│   ├── NavBar.vue
│   ├── CarouselBanner.vue
│   ├── ProductCard.vue
│   └── ProductDetails.vue
├── router/
│   └── index.js
└── App.vue
```

Routes

Path	View	What it does
/	HomeView.vue	Carousel banner + product cards grid
/product/:id	ProductView.vue	Single product detail + related products
/about	AboutView.vue	Static about page

Components

Component	Responsibility
NavBar.vue	RouterLink navigation — Home, Product, About
CarouselBanner.vue	DaisyUI carousel, minimum 3 slides, hardcoded images fine
ProductCard.vue	Product image, name, price, badge — navigates to detail on click
ProductDetails.vue	Full product info, Buy Now button, emits buy event up

Data Shape — App.vue Only

Define the following array in App.vue. Pass it down to views and components via props. Do not modify the structure.

```
[
  {
    "id": 1,
```

```

    "name": "Cozy Sneakers",
    "description": "High-quality sneakers that go with everything you wear.",
    "image": "https://via.placeholder.com/300",
    "badge": "NEW",
    "price": 120,
    "discount": 20,
    "stock": 10,
    "tags": ["Fashion", "Casual", "Sport"]
  },
  {
    "id": 2,
    "name": "Running Shoes",
    "description": "Built for speed and comfort on any terrain.",
    "image": "https://via.placeholder.com/300",
    "badge": "",
    "price": 90,
    "discount": 10,
    "stock": 5,
    "tags": ["Sport", "Running"]
  },
  {
    "id": 3,
    "name": "Casual Boots",
    "description": "Rugged boots for everyday adventures.",
    "image": "https://via.placeholder.com/300",
    "badge": "SALE",
    "price": 150,
    "discount": 0,
    "stock": 8,
    "tags": ["Casual", "Winter"]
  },
  {
    "id": 4,
    "name": "Flip Flops",
    "description": "Light and breezy for sunny days.",
    "image": "https://via.placeholder.com/300",
    "badge": "",
    "price": 30,
    "discount": 50,
    "stock": 20,
    "tags": ["Summer", "Casual"]
  }
]

```

What Each File Does

HomeView.vue

- Renders CarouselBanner at the top
- Renders a grid of ProductCard components from the products prop
- Receives products array as prop from App.vue

- onMounted — log: `"HomeView mounted - X products loaded"`
- onUnmounted — log: `"HomeView unmounted"`

CarouselBanner.vue

- A DaisyUI carousel with minimum 3 banner slides
- Hardcoded banner images are acceptable — this is UI only
- onMounted and onUnmounted with console logs

ProductCard.vue

- Receives a single product object as prop
- Displays image, name, price, badge
- View Product button navigates to the product detail page without reload
- onMounted and onUnmounted with console logs

ProductView.vue

- Dynamic route — reads the product ID from the route on mount
- Finds the matching product from the products prop
- Passes matched product to ProductDetails via prop
- Related products = all products except the current one, rendered as ProductCard components
- onMounted — log: `"ProductView mounted for ID: X"`
- onUnmounted — log: `"ProductView unmounted"`

ProductDetails.vue

- Receives product as prop
- Displays full product info — image, name, description, price, discounted price, stock, tags
- Buy Now button emits `'buy'` event with product ID up to parent
- If stock is 0 — disable the button and show Out of Stock
- onMounted and onUnmounted with console logs

AboutView.vue

- Static page — your name, the course name, ITI
- One paragraph about the store
- onMounted and onUnmounted with console logs

NavBar.vue

- Links to Home, a product page, and About
- RouterLink only — never an <a> tag
- onMounted and onUnmounted with console logs

Communication Flow

The Golden Rule

Props go down. Events go up. App.vue owns all the data. Nobody else touches it directly.

```
App.vue (owns products array + handles stock decrease)
|
| :products  —————> prop down
|
▼
HomeView.vue / ProductView.vue
|
| :product  —————> prop down
|
▼
ProductDetails.vue
|
| $emit('buy', productId)  —————> event up
|
▼
ProductView.vue
|
| $emit('buy', productId)  —————> event up (relay)
|
▼
App.vue → decreases stock → prop updates → UI re-renders
```

Acceptance Criteria

#	Criteria
1	Navigating between pages never causes a full page reload
2	Console shows correct mount and unmount logs for every component as you navigate
3	Clicking a product card navigates to the correct product detail page
4	Refreshing /product/2 in the browser still shows the correct product

#	Criteria
5	Buying a product decreases stock and the UI reflects instantly
6	Stock reaching zero disables the buy button and shows Out of Stock
7	Related products shows all products except the current one
8	About page is reachable from the navbar

Bonus Challenge

Add a live stock counter in `NavBar.vue` showing total items in stock across all products. It must update automatically when a purchase is made — no manual refresh. The data still lives in `App.vue` and flows down to the navbar as a prop.

Pinia and composables will be covered in Day 3.